

Low-Complexity and Low-Energy RNN-Enhanced Kalman Filter for Channel Estimation: A Case Study

ANTOINE SIEBERT^{1,2,3}, BERTRAND LE GAL¹, GUILLAUME FERRE², AND AURÉLIEN FOURNY³

¹IRISA/Inria Laboratory, UMR CNRS 6074, University of Rennes, 35000 Rennes, France

²CNRS, Bordeaux INP, IMS, UMR 5218, University of Bordeaux, 33400 Talence, France

³Thales Six GTS France, 49300 Cholet, France

CORRESPONDING AUTHOR: GUILLAUME FERRE (guillaume.ferre@u-bordeaux.fr)

ABSTRACT In radiocommunications, multipath propagation often induces Inter-Symbol Interference (ISI), which requires accurate channel estimation and equalization. Neural networks have recently shown promising performance for this task but remain limited by their high computational cost and lack of interpretability, hindering their adoption in mission-critical systems. To overcome these limitations, this study proposes a hybrid estimation architecture that combines a linear Kalman filter with two lightweight neural networks that dynamically and deterministically adjust the noise covariance matrices of the filter, enabling efficient and explainable channel tracking. Experimental results show performance gains of up to 5 dB at low SNR and 3 dB at high SNR compared with the Least Squares (LS) algorithm. This low-dimensional design enables real-time deployment on energy-constrained programmable platforms. Implementations on an ARM Cortex-A72 CPU and Artix-7/Virtex-7 FPGAs demonstrate throughputs of 53.7 Mbps and 8-20 Mbps, respectively, with FPGA energy consumption over $20\times$ lower than on the CPU. These results confirm the effectiveness of the proposed NN-enhanced Kalman estimator for interpretable and energy-efficient channel estimation in embedded and defense applications.

INDEX TERMS Channel estimation, digital communication, neural networks, Kalman filter, Rayleigh channel, hardware implementation, high-level synthesis, FPGA.

I. INTRODUCTION

IN WIRELESS digital communication, the surrounding environment significantly influences signal transmission. Owing to physical phenomena, such as reflection, diffraction, and scattering, the received signal at any given time consists of multiple delayed replicas of the transmitted signal, each with different power levels. This phenomenon is known as multipath propagation [1]. Multipath propagation introduces Inter-Symbol Interference (ISI), which must be mitigated or eliminated to ensure accurate symbol detection on the receiver side. The process of compensating for ISI is known as equalization [2]. Effective equalization requires an accurate estimation of the propagation channel, which can often be modeled as a linear filter fully characterized by its impulse response $\mathbf{h}$. In this scenario, pilot symbol sequences known *a priori* by both the transmitter and the receiver are commonly employed to estimate the Channel Input Response (CIR) [3]. A large set of estimation techniques at the receiver side has been proposed and applied in real systems depending on the channel characteristics, targeted computational complexity, and/or memory consumption. Traditionnal system

may employ the Least Squares (LS) algorithm [4], Least Mean Squares (LMS), Recursive Least Squares (RLS) [5] or Kalman Filter (KF) [6], [7], [8].

Recently, deep learning approaches targeting the physical layer of communication systems have been proposed for modulation recognition [9], coding and decoding [10], and channel estimation [11], [12]. For instance, for 5G and 6G communication systems, a large set of studies targeting estimation challenges has been proposed, such as CNN [13], [14], GAN [15], [16], Long Short Term Memory (LSTM) [15], [17] and Recurrent Neural Network (RNN) [13]. All of these neural networks have demonstrated remarkable effectiveness in capturing complex temporal dependencies and extracting meaningful features from sequential data. However, they mainly focus on estimation performance, disregarding their real-time implementation possibility or the explainability needed, for instance, in critical systems. Indeed, the use of neural networks in safety-critical embedded and connected systems poses several non functional challenges, particularly in terms of security, explainability, and integration feasibility.

This paper presents the Smart Kalman Filter (SKF) approach introduced in [18], which is a hybrid channel estimation algorithm based on a linear Kalman filter combined with two neural networks and their implementation characteristics. The SKF architecture was designed with particular attention to explainability, estimation performance, and implementation efficiency. The contributions of this paper can be summarized as follows:

- We present the SKF channel estimation approach, explaining the neural network design choices performed to maintain explainability and low-complexity features.
- We present a fair comparison of the proposed approach against solutions included in real communication systems.
- We report a set of experimental results, including energy measurements, on the CPU and FPGA targets, demonstrating its real-time viability.

The remainder of this paper is organized as follows: Section II reviews the state of the art and outlines the motivations behind our work. Section III introduces the proposed approach. In Section IV, the proposed method is compared with common solutions from the literature. Implementation strategies for real-time execution, as well as experimental results obtained on embedded CPU and FPGA platforms, are detailed in Section V. Finally, the research perspectives and conclusions are reported in Section VI.

II. RELATED WORKS

Data equalization [2] is a critical step in digital communication systems, because it directly affects the demodulation performance and, consequently, the overall reliability of the communication link. Effective equalization relies on accurate estimation of the propagation channel. The channel can often be modeled as a linear filter fully defined by its impulse response $\mathbf{h}$. To facilitate Channel Impulse Response (CIR) estimation, a commonly used technique consists of inserting pilot symbol sequences known in advance by both the transmitter and receiver [3]. On the receiver side, many estimation techniques from the literature can be applied in practical systems, depending on the available computational resources and memory constraints, to extract channel parameters. Among the best known techniques, we can cite the Least Squares (LS) method [4], Least Mean Squares (LMS), Recursive Least Squares (RLS) algorithms [5] or more advanced approaches based on Kalman filtering [7], [8], [19], [20], [21]. Indeed, because the effect of the propagation channel can be mathematically modeled as a convolution, the Kalman Filter (KF) naturally arises as a suitable estimation technique [6]. This technique was used to track time-varying parameters in linear systems and is optimal in the Minimum Mean Square Error (MMSE) sense under certain assumptions. Its use extends well beyond telecommunications and has been successfully applied in domains such as ballistic-target tracking [19]. Although KF-based channel estimation has been extensively studied [20], [21], its practical

deployment in real-world communication systems remains limited. This is largely due to its sensitivity to parameter tuning: a filter calibrated for one scenario may perform poorly in another. To overcome this limitation, an Interacting Multiple Model (IMM) approach has been proposed [22]. The IMM combines several Kalman filters with distinct parameter sets, enabling the system to dynamically adapt to changing conditions, albeit at the cost of increased computational complexity. Despite this improvement, the need for manual parameter tuning, both at design time and runtime, continues to represent a significant challenge.

Recent advances in deep learning have increasingly influenced the physical layer of communication systems, enabling significant progress in areas such as modulation recognition [9], channel coding and decoding [10], and channel estimation [11], [12]. Among the various architectures explored, Recurrent Neural Networks (RNNs) have proven particularly effective in modeling complex temporal dependencies and extracting relevant features from sequential data, making them well-suited for dynamic signal processing tasks. Several studies have investigated the integration of Kalman filters and neural networks (NNs) for signal and signal processing applications. The authors of [23] and [24] employed recurrent neural networks to estimate filter parameters in ballistic and pose estimation scenarios from images. These methods rely on Long Short-Term Memory (LSTM) layers, which are unsuitable for embedded real-time constraints owing to their high computational cost.

The authors of [25] propose an alternative that estimates the Kalman gain directly via a neural network to estimate vehicle trajectory correction, thus avoiding explicit parameter tuning. However, this approach undermines the optimality of the filter and exacerbates the closed box nature of the system. Consequently, it is not directly suitable for military communications in which model transparency and reliability are critical. Despite their promising performance, deep learning-based methods have several limitations. First, their lack of interpretability poses a challenge in critical applications where decision traceability is essential. Second, their high computational and memory demands hinder their deployment in resource-constrained environments such as embedded or real-time systems.

Recent advances in digital communication systems have coincided with the emergence of platforms that support real-time inference of neural networks (NN). Although high-performance AI models can now be deployed for complex tasks [26], [27], [28], they typically rely on distributed GPU-based infrastructures [29], [30]. To enable their use on conventional processors or embedded systems with limited resources, research has focused on reducing model complexity [31], [32], optimizing network architectures [33], and minimizing computational and memory requirements [34].

Among other execution platforms, FPGA-based implementations are a promising alternative, offering greater energy efficiency compared with CPU or GPU solutions [35],

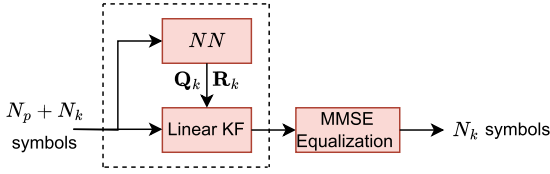


FIGURE 1. General principle of the Smart Kalman filter.

[36], [37]. However, their design and deployment remain complex and time-consuming, often requiring advanced optimization strategies such as pruning and quantization [38], [39], [40]. These techniques improve compatibility with FPGA resources while preserving inference accuracy and satisfying the strict latency and power constraints inherent in real-time applications.

Despite the effectiveness of these optimization and implementation methodologies on target CPUs and FPGAs, it should be noted that they cannot replace the choices and optimizations made at the algorithmic level to limit the computational and memory complexity of the models.

In this paper, we present a hybrid channel-estimation algorithm, called the Smart Kalman Filter (SKF), based on neural networks and a linear Kalman filter, as illustrated in Figure 1. Integrating neural networks with Kalman filtering offers a promising hybrid approach in which deep learning can dynamically adjust KF parameters over time using the N_p symbols of pilot sequences, making the filter more robust for correcting the N_k payload symbols. Contrary to previous studies, particular attention is devoted to (1) maintaining the explainability feature and (2) keeping the computation and memory complexities of the involved NN structures low.

III. SMART KALMAN FILTER

In this section, we first present the notations used in Kalman filters and then detail the SKF approach and the choices made in terms of network structures to retain explicability and low-complexity features.

A. NOTATIONS AND CONTEXT

Before providing details about the Smart Kalman Filter, the notation is introduced, and a brief context regarding the Kalman filter is provided. In this study, we use the following notations:

- $\mathbf{x}$ represents either a vector or a matrix,
- $\mathbf{x}_k$ denotes the state of the vector $\mathbf{x}$ at time step k ,
- $x(n)$ refers to the n^{th} element of the vector $\mathbf{x}$,
- $x(i, j)$ corresponds to the entry located at row i and column j of the matrix $\mathbf{x}$,
- $\mathbf{x}^T$ indicates the transpose of $\mathbf{x}$,
- L is the length of the channel impulse response (CIR),
- N represents the length of the CIR estimators, with $N \geq L$,
- N_p denotes the number of pilot symbols per transmission packet.
- $\Re$ and $\Im$ refer respectively to the real and imaginary parts.

The system under consideration follows a linear state-space model, that evolves over time according to:

$$\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k \quad (1)$$

where $\mathbf{x}_k$ is the state vector at time k , $\mathbf{F}_k$ is the state transition matrix governing the system dynamics, $\mathbf{B}_k$ is the control matrix applied to input vector $\mathbf{u}_k$, and $\mathbf{w}_k$ is the process noise.

The KF is composed of two main steps: a prediction phase and an update phase. The prediction equations are as follows:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}_k \hat{\mathbf{x}}_{k-1|k-1} + \mathbf{B}_k \mathbf{u}_k \quad (2)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k \quad (3)$$

where $\hat{\mathbf{x}}_{k|k-1}$ is the predicted state estimate at time k based on past observations, and $\hat{\mathbf{x}}_{k-1|k-1}$ is the corrected estimate at time $k-1$. The term $\mathbf{P}_{k|k-1}$ represents the predicted covariance matrix of the state estimate and $\mathbf{Q}_k$ is the noise covariance matrix of the process, defined as $\mathbf{Q}_k = \mathbb{E}[\mathbf{w}_k \mathbf{w}_k^T]$.

During the update step, the filter integrates the new measurements to refine the state estimate. This process is described by the following equations:

$$\tilde{\mathbf{y}}_k = \mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_{k|k-1} \quad (4)$$

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k \quad (5)$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1} \quad (6)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \tilde{\mathbf{y}}_k \quad (7)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \quad (8)$$

where $\mathbf{z}_k$ is the measurement vector at time k and $\mathbf{H}_k$ represents the observation matrix that relates the state vector to the measurements. The term $\tilde{\mathbf{y}}_k$ is called innovation and measures the discrepancy between actual and predicted observations. The covariance of the innovation is given by $\mathbf{S}_k$, and $\mathbf{R}_k$ is the measurement noise covariance matrix. The Kalman gain, denoted by $\mathbf{K}_k$, determines the weight assigned to new measurements in the state update. Finally, the identity matrix is represented by $\mathbf{I}$. One of the main challenges in the practical implementation of the Kalman filter is the proper selection of $\mathbf{Q}_k$ because it does not have a direct physical interpretation. $\mathbf{Q}_k$ is often manually tuned based on the application. If the values of $\mathbf{Q}_k$ are too large, the estimated state will exhibit excessive variations, leading to tracking instability. Conversely, if $\mathbf{Q}_k$ is underestimated, the filter may respond too slowly to changes in the system, potentially resulting in significant estimation errors. To address this limitation, we propose a deep learning-based approach to dynamically estimate the covariance matrices $\mathbf{Q}_k$ and $\mathbf{R}_k$ at each time step k , allowing an adaptive Kalman filter. The proposed architecture is illustrated in Figure 2.

The Smart Kalman Filter proposed to estimate the CIR using a KF was enhanced by two neural networks, denoted as NN_1 and NN_2 . They determine two key parameters at each time step k : the measurement noise covariance matrix ($\mathbf{R}_k$) and the process noise covariance matrix ($\mathbf{Q}_k$). These estimations rely on the last p pilot sequences received, each with

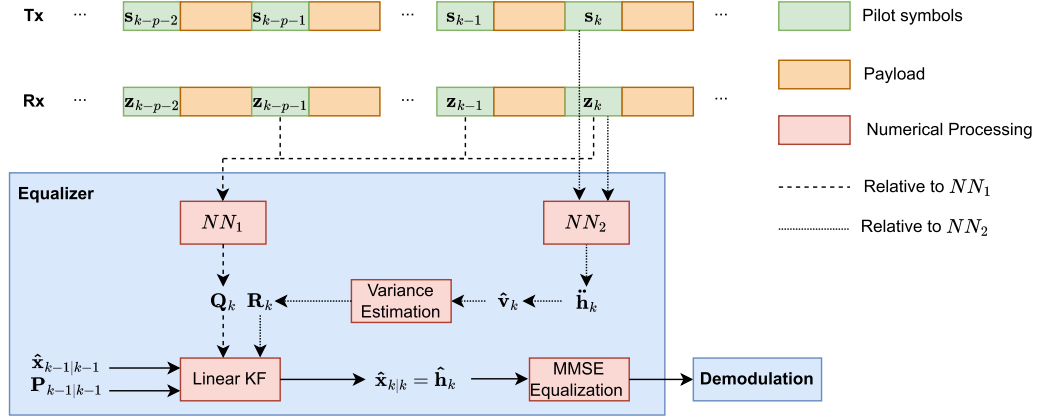


FIGURE 2. Principle of the proposed architecture: The Smart Kalman filter.

length N_p . Note that the transmitted pilot sequence consisting of N_p symbols remained identical across all packets.

Let $\mathbf{h}$ be the impulse response of the channel and $\mathbf{s}$ the transmitted pilot sequence. The observation at time k is:

$$z_k(i) = \sum_{j=0}^{L-1} h(j)s(i-j) + v(i) \quad (9)$$

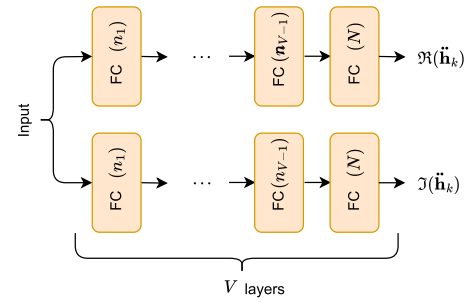
where $\mathbf{v}_k = [v(0), \dots, v(N_p - 1)]$ is the additive noise such as $\mathbf{v}_k \sim \mathcal{CN}(0, \sigma_v^2 \mathbf{I}_{N_p})$. Given that, the goal is to estimate the CIR, we define the state vector as follows:

$$\mathbf{x}_k = [h_k(L-1), h_k(L-2), \dots, h_k(0)]^T \quad (10)$$

The observation can then be rewritten as $\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$ with $\mathbf{H}_k$ being the observation matrix composed of the transmitted pilot symbols. In the context of CIR estimation, for all $k \in \mathbb{N}$, the control-input matrix $\mathbf{B}_k$ is set to zero, and the initial covariance matrix $\mathbf{P}_0$ is initialized as the identity matrix. The state transition matrix $\mathbf{F}$ was also set as the identity matrix because of the unknown evolution of the CIR. To complete the Kalman filter formulation, noise covariance matrices $\mathbf{R}_k$ and $\mathbf{Q}_k$ must be specified.

B. ESTIMATION OF THE COVARIANCE MATRICES

Neural networks $\mathbf{NN}_1$ and $\mathbf{NN}_2$ were introduced to estimate the covariance matrices. To estimate the first covariance matrix $\mathbf{R}_k$ at each time k , a preliminary estimation $\ddot{\mathbf{h}}$ of the CIR was obtained using a neural network denoted as $\mathbf{NN}_2$. Because the propagation channel is convolutional in nature, linear layers were used to build the $\mathbf{NN}_2$ neural network. Network $\mathbf{NN}_2$ is a Multilayer Perceptron (MLP) composed of two branches of V layers. The network structure is illustrated in Figure 3. The input to $\mathbf{NN}_2$ consists of the real and imaginary parts of the transmitted and received pilot symbols, resulting in a total input size of $2 \times 2 \times N_p$. The network outputs the real and imaginary components of the CIR, each of size $1 \times N$, where $N \geq L$. At time k , an estimate $\ddot{\mathbf{h}}_k$ of the channel impulse response was obtained. The observation noise is then

FIGURE 3. General structure of the neural network $\mathbf{NN}_2$. It is composed of two branches of V fully connected layers.

estimated as:

$$\hat{v}(i) = \sum_{j=0}^{L-1} \ddot{h}_k(j)s(i-j) - z_k(i) \quad (11)$$

The associated noise variance is obtained using the unbiased estimator:

$$\sigma_v^2 = \frac{1}{N_p - 1} \sum_{i=0}^{N_p-1} \left[(\Re(\hat{v}(i) - \bar{v}))^2 + (\Im(\hat{v}(i) - \bar{v}))^2 \right],$$

where $\bar{v}$ is the empirical mean of $\hat{v}$, and N_p is the number of pilot symbols in each pilot sequence. The observation noise covariance matrix is then defined as $\mathbf{R}_k = \sigma_v^2 \mathbf{I}$ and can be updated at each time step if required.

The second covariance matrix $\mathbf{Q}_k$ of the KF is the process noise covariance matrix. This represents the degree of variation in the state vector from the evolution model at each step. By considering the independence between the different CIR coefficients, covariance matrix $\mathbf{Q}_k$ becomes:

$$\mathbf{Q}_k = \text{Diag}(\sigma_0^2, \dots, \sigma_{N-1}^2) \quad (12)$$

For each $i \in \{0, \dots, N-1\}$, σ_i^2 represents the variance of the i^{th} coefficient. To estimate $\mathbf{Q}_k$ over time, a second neural network is used, which is denoted $\mathbf{NN}_1$ and is represented

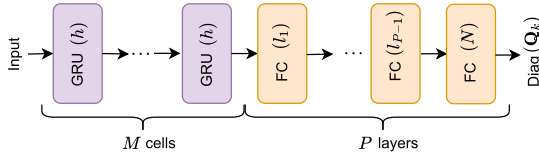


FIGURE 4. General structure of the neural network NN_1 . It is composed of M GRU layers followed by P fully connected layers.

in Figure 4. The network must be able to learn the temporal dependencies between the received data. To achieve this, the first layers of the network are M Gated Recurrent Unit (GRU) [41] layers of size h . This type of layer allows the extraction of important features from the input vector over time and synthesizes them into a hidden vector. The GRU layers were followed by P fully connected layers of sizes $l_0, \dots, l_{P-1}$. This number is subject to change, depending on the estimation context. The precise structure of the network and the numbers of layers M and P are discussed in Subsection III-C. This architecture is based on [24], with an LSTM network replaced by a GRU to reduce computational complexity.

C. COMPUTATIONAL COMPLEXITY OPTIMIZATION

Before proceeding with the SKF performance evaluation, the configuration of the NN_1 and NN_2 neural networks is detailed. For each, multiple architectures have been developed and tested to ensure their compatibility with an embedded implementation. The aim of this study was to develop lightweight, high-performance structures. To find the best tradeoff solution, experiments were conducted on the number of layers and their sizes.

For network NN_1 illustrated in Figure 4, the study of the number of GRU cells M and associated hidden vector sizes within the ranges $[1, 8]$ and $[2, 1024]$ showed that increasing these parameters did not yield a significant performance improvement after training and simulation. Consequently, a single GRU layer was selected. Furthermore, an analysis of the cell dimensions revealed that exceeding the threshold of 128 no longer enhanced the obtained performance, while increasing the computational complexity. The GRU layer was followed by P fully-connected layers, the configuration of which was also investigated. The size of the final layer was set to $N = 5$, in accordance with the network output dimensions, corresponding to the number of taps of the considered channel. After experimenting with the number of fully connected layers P , which varied from 1 to 5 with decreasing sizes from 1024 to 5, the network architecture was fixed to three layers of sizes 32, 16, and 5. This configuration provided the best performance-complexity trade-off for our dataset. Activation functions were also evaluated. A sigmoid function was selected to bind the network output between 0 and 1, to prevent divergence during training and ensure realistic magnitudes for the coefficients of $\mathbf{Q}_k$ in the given context. To obtain results and converge to the presented sizes, multiple neural networks were trained on channels with small and

large variations between two consecutive packets to adapt to all situations in the context of slow-fading channels. Therefore, the channels that are used must be temporally correlated. There are several such channels, but the Rayleigh channel correlated with the Jakes spectrum is one of the most used in the absence of a line of sight between the transmitter and the receiver. There are multiple ways to generate this type of channel, based on the inverse Fourier transform [42], white noise filtering or based on a sum of sinusoids. For fast implementation, the channels are generated according to the sinusoidal method:

$$h_k(i) = \frac{1}{\sqrt{N_e}} \sum_{j=0}^{N_e-1} e^{j(\phi_{i,j} + 2\pi f_d \cos(\theta_{i,j})kT_s)} \quad (13)$$

with $N_e = 32$ the number of sinusoids, and $\forall i \in \llbracket 0; N-1 \rrbracket$, $\forall j \in \llbracket 0; N_e-1 \rrbracket$, $\phi_{i,j} \sim \mathcal{U}(0, 2\pi)$ and $\theta_{i,j} \sim \mathcal{U}(0, 2\pi)$. The maximum Doppler frequency is denoted f_d . In the following section, we discuss the maximum normalized Doppler frequency $\tilde{f}_d = f_d \times T_s$, where T_s is the duration between two consecutive symbols.

Training was carried out on a dataset of size 2×10^6 channels, which was divided into 80% for training and 20% for testing. Each channel of the dataset contains $L = 5$ multipaths, and the normalized Doppler frequency $\tilde{f}_d$ changes independently and uniformly on each path for all 100 packets in $\{10^{-3}, 10^{-2}, 10^{-1}\}$ to create abrupt variations, observable in our military context. These Doppler frequencies correspond to the standard HF/UHF vehicle speeds.

Similar studies have been conducted for the design of network NN_2 , structured into two parallel branches, each composed of V fully connected layers with sizes $n_1, \dots, n_{V-1}$ as shown in Figure 3. The first branch estimates the real part of the propagation channel whereas the second branch estimates the imaginary part. The initial layers in each branch used the LeakyReLU activation functions, whereas the final layer had no activation. Because the channel coefficients are unbounded *a priori*, functions such as Rectified Linear Unit (ReLU), hyperbolic tangent, or sigmoid are unsuitable. After experimenting with V ranging from 1 to 5, the results demonstrated that three fully connected layers of sizes 128, 32, and 5 respectively, were sufficient to reach the best channel estimate, yielding an effective estimation of the matrix $\mathbf{R}_k$. Increasing the layer size only marginally improved the accuracy at the cost of a much higher computational complexity.

To be as general as possible, the training dataset of this neural network must be composed of several different CIR, for either the channel depth or coefficient amplitude. The dataset used for the training of NN_2 was composed of 5×10^4 vectors of size $1 \times N$. Each vector coefficient was generated according to a complex Gaussian distribution ($\mu=0, \sigma^2=1$) representing the CIR. The length L of the CIR is uniformly distributed between one and five, and the vector is zero-padded to obtain a size of $N = 5$. Zero padding was used to have a fixed vector length to facilitate the training of the neural network. The dataset was divided into two parts: 80% for training and 20%

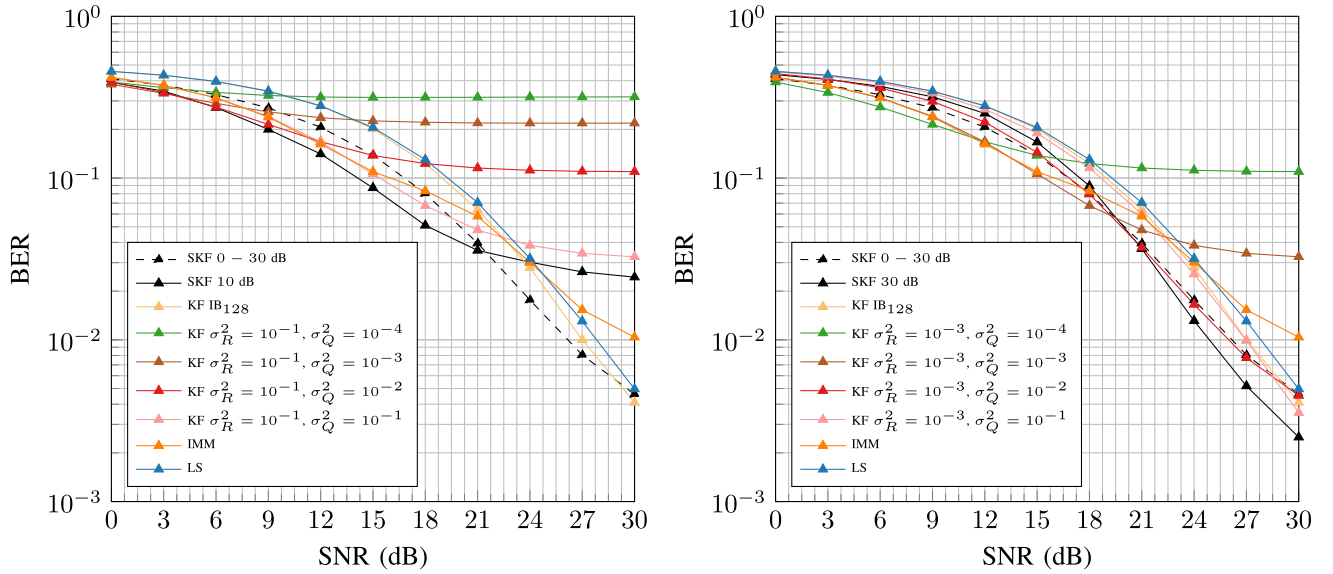


FIGURE 5. BER comparison between multiple channel estimation methods; the proposed architecture (SKF) and others conventional algorithms.

for testing. The batch size was 128 and the Adam optimizer and MSE loss were used. The MSE loss is given by:

$$MSE(\hat{\mathbf{h}}, \mathbf{h}) = \frac{1}{N} \sum_{j=0}^{N-1} (\Re(\hat{h}(j) - h(j))^2 + \Im(\hat{h}(j) - h(j))^2) \quad (14)$$

Using the training procedures for both neural networks described, we review the obtained performance results in the next section.

IV. PERFORMANCE EVALUATION

This section is divided into two parts. The first part compares the performance of the proposed system with conventional approaches. The second part of the section investigates the impact of data quantization on performance.

A. COMPARISON WITH RELATED WORKS

Simulations were performed to assess the Bit Error Rate (BER) performance of the proposed architecture. The simulation parameters are presented in this section. The performance of the proposed Smart Kalman Filter is compared with that of an IMM composed of four Kalman filters parameterized to perform at any SNR level, multiple linear Kalman filters, and LS algorithms over Rayleigh fading multipath channels. In addition, a linear Kalman filter with an innovation-based method for updating the covariance matrices was considered [43]. No comparison is made with related works on hybrid Kalman filters [23], [24], [25]. Indeed, those works are dedicated to the fields of signal and image processing (motion and position estimation), making any comparison impossible. Similarly, even though experiments were conducted to adapt the techniques presented in [11] and [12] for OFDM and MIMO communication schemes,

the lack of explainability of these solutions and the irrelevant results motivated us not to present them.

In the following simulations, packets of 256 symbols were transmitted, each containing a pilot sequence of $N_p = 5$ known symbols at the beginning, which was less than 2% of the packet. The remaining $N_k = 251$ symbols carry data. QPSK modulation was employed. This packet structure is inspired by a reference industrial waveform. The channel impulse response was assumed to be constant within each packet and varied only between packets. A slow-fading channel was considered. Perfect time and frequency synchronizations were considered on the receiver side. A linear MMSE equalizer of length 15 was used with no channel coding. To estimate the $\mathbf{Q}_k$ matrix over time, the number of pilot sequences was set to $p = 2$, because tests with $p \geq 3$ did not yield performance gains.

Figure 5 presents the final performance of the SKF in terms of BER for the different tested strategies. The left subfigure presents an application context in which the targeted SNR was 10 dB where the SKF was trained at 10 dB. The right subfigure shows the performances of the approaches for higher SNR values. From the figures, it can be observed that SKF trained at 10 dB outperformed the other algorithms at low SNR values. This trend was similar when simulations with high SNR training values were examined. For a BER of 10^{-1} , SKF offers a gain of approximately 2 dB over the best KF, and approximately 5 dB over the LS algorithm. The SKF trained at 30 dB provided a BER of 10^{-2} a gain of approximately 3 dB over the LS algorithm and approximately 2 dB over the best KF solution. It is noted that the KF with a covariance matrix $\mathbf{Q}_k$ having low diagonal coefficients performs worse than the others, as the filter struggles to track rapidly varying channels. We also observed that IMM offers a good compromise across all SNR levels, although it is never

the best solution. Finally, we also investigated the impact on performance of SNR specialization during SKF training. To do this, a generic SKF model was trained on a dataset in which the SNR ranged from 10 to 30 dB in order to compare performance levels with specialized SKFs. The BER curves for these models are presented in Figure 5. Unfortunately, despite good performance levels in both experiments, these BER curves show that the generic version is less suitable for the targeted SNR operating points.

B. DATA QUANTIZATION

The proposed SKF approach was designed and optimized at the algorithmic level to simplify its implementation. However, before presenting the implementation results, the choices made for data and computation quantization to minimize memory footprint and computational complexity are detailed. To minimize the computational cost and memory complexity, previous NN models trained and evaluated using floating-point weights and computations should be quantized [32], [44], [45], [46], [47]. This step lowers the weight precision and limits the dynamic range of operations, at the cost of a potential accuracy degradation. Optimization opportunities for programmable targets (CPUs) are often constrained by data formats and arithmetic operations natively supported by these platforms [48]. In contrast, on hardware targets (ASICs/FPGAs), the design space is less restricted, allowing for finer adjustments when model performance must be preserved [49]. Note that owing to system-level constraints, input samples produced by the analog-to-digital converter (ADC) must be quantized using a 16-bit fixed-point representation.

Because the two neural networks NN_1 and NN_2 are independent, their optimization was performed separately to reduce the overall design space. The most promising solutions were then combined and evaluated to identify effective pairings. Furthermore, the post-training optimization of the weight quantization format was adopted to limit the complexity of the quantization process. For the implementation, all neural network weights were converted to fixed-point integers using a conventional approach, to simplify the additions and multiplications during inference. In the following, $\langle n, m \rangle$ refers to a quantization where n bits are used in the integer part and m bits for the fractional part. A large set of bitwidth configurations was tested to assess the trade-offs between computational complexity and performance.

In the second phase, the quantification of sigmoid and hyperbolic tangent activation functions is considered. To facilitate potential software and hardware implementations, it was decided to implement them as Look-Up Tables (LUT), meaning that the depth and width of the table must be fixed. To determine the ideal quantization interval, the output interval of the values obtained using the float32 neural network was analyzed. The values were in the range $[10^{-5}, 0.5]$. Thus, the following interval limits were considered:

$$\sigma(-10) \approx 4.54 \times 10^{-5} \quad (15)$$

$$\sigma(-11) \approx 1.67 \times 10^{-5} \quad (16)$$

$$\sigma(-12) \approx 6.14 \times 10^{-6} \quad (17)$$

Because the FPGA memory blocks have a width of 18 bits, this encoding format was selected: using one bit for the sign and one bit for the integer part. Because $2^{-16} \approx 1.53 \times 10^{-5}$ is slightly less than $\sigma(-11)$, the quantization interval was chosen to be equal to $[-11, 11]$. The same quantization strategy was applied for the hyperbolic tangent but was adapted for an output interval in the range $[-1; 1]$.

Figure 6 illustrates the post-equalization performance when quantization is applied to either NN_1 or NN_2 , while keeping the other network in floating-point precision. The curves show the BER as a function of the weight quantization formats. It can be observed from the BER curves in Figure 6(a) that the sigmoid and hyperbolic tangent functions, when quantized to 18-bit precision using 128 levels, maintain a BER that is comparable to that of the unquantized version.

In addition, experiments on NN_1 show that its weights can be quantized to 8 bits without any loss in performance compared to the floating-point model. The curves in Figure 6(b) further highlight that the network NN_2 requires a higher bitwidth to preserve its original floating-point performance. Unlike GRUs, fully connected networks lack the temporal dynamics to mitigate errors, leading to a direct and linear accumulation of quantization errors across layers. Specifically, performance degradation at high SNR becomes noticeable when the weights are quantized below 11 bits.

Finally, the impact of quantizing both networks on the overall estimation and equalization performance is assessed. As shown in Figure 7, using the optimal quantization formats for both NN_1 and NN_2 does not introduce any observable degradation. The configuration employing 8-bit quantization for NN_1 and 12-bit quantization for NN_2 was therefore selected and adopted in the remainder of this work.

V. EXPERIMENTATION RESULTS

In the previous sections, we indicated that the NN_1 and NN_2 networks had low computational complexity compatible with their real-time deployment in constrained embedded systems such as those available in military applications. In this section, we present the results of characterizing the networks on low-end and mid-range CPU and FPGA targets. The waveform used in the experimental section is identical to that previously described, with 256 symbols ($N_p = 5$ and $N_k = 251$), and corresponds to the military context in which this work was conducted.

A. SOFTWARE-BASED IMPLEMENTATIONS

Currently, for flexibility reasons, all or part of the communication systems can be deployed as software in multi-core architectures owing to their large processing performance. Consequently, three hardware platforms composed of such CPUs were selected to represent different use cases in terms of usage contexts. Their characteristics are listed below:

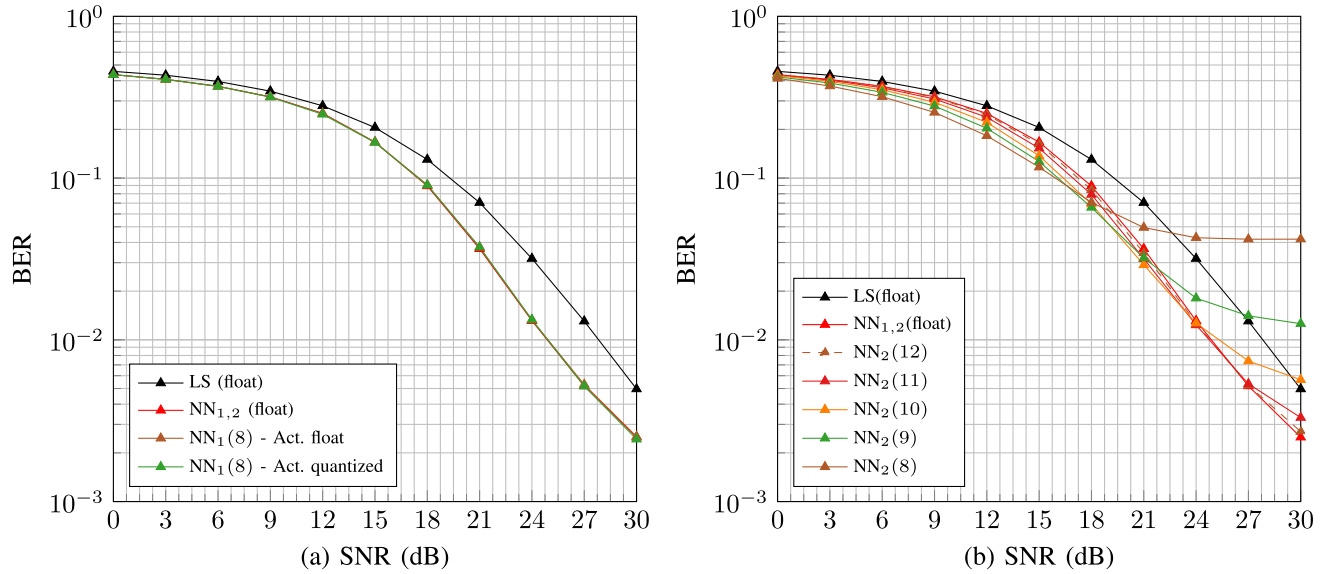


FIGURE 6. Impact of NN_1 and NN_2 quantization on BER performances. (a) performances curves when NN_2 is not quantized, (b) performances curves when NN_1 is not quantized.

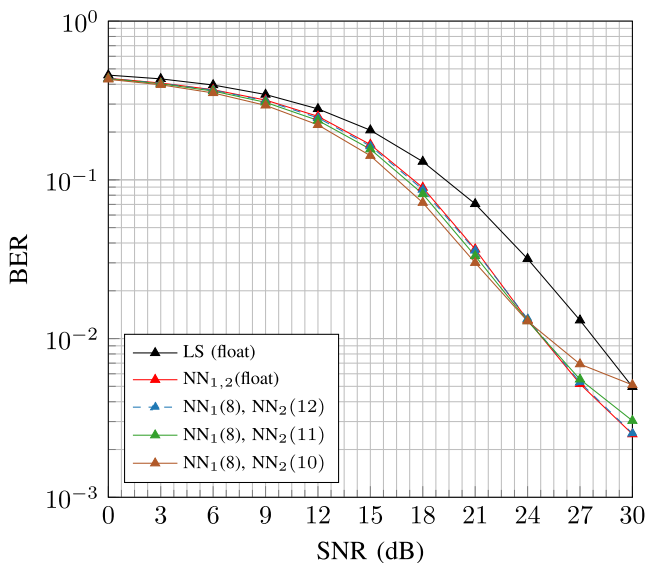


FIGURE 7. BER performances post-egalization stage when NN_1 and NN_2 are fully quantized depending on the bitwidth.

- **Platform 1 ($\mathcal{P}_1$)** was a low-performance ARM Cortex-A72 CPU with a frequency of 1.5 GHz. The power consumption was ≈ 1 W in single-core mode.
- **Platform 2 ($\mathcal{P}_2$)** was composed of a high-end Apple M1 Pro CPU. Its high-performance cores have a CPU clock frequency of 3.2 GHz whereas the power consumption was ≈ 7 W in a single-core setup.
- **Platform 3 ($\mathcal{P}_3$)** was composed of an INTEL i7 12700T CPU with a clock frequency of up to 4.70 GHz. The measured power consumption was ≈ 33 W.

The selected processor cores have SIMD units [50], [51], that are particularly effective at vectorizing and therefore

TABLE 1. Throughput and energy characteristics of the NN_1 and NN_2 CPU-based implementations.

Platform	Processor frequency	Execution (3000 i/s)	Maximum throughput	Energy per bit
$\mathcal{P}_1$	1.50 GHz	152.61 ms	4.95 Mbps	202 ηj
$\mathcal{P}_2$	3.2 GHz	14.77 ms	53.78 Mbps	130 ηj
$\mathcal{P}_3$	4.70 GHz	16.06 ms	47.06 Mbps	701 ηj

accelerating neural network calculations, whether for training or inference. In this work, neural networks were implemented in the C programming language to explicitly expose the computational parallelism between the layers of the network. The intrinsic vector functions provided by INTEL and ARM were applied to exploit fine-grained parallelism during execution [52], [53].

Table 1 summarizes the performance of the fixed-point implementations in terms of maximum throughput and power consumption. Execution times were measured based on the processing of 3,000 pilot sequences, corresponding to a target information throughput of 753 kbps. In parallel, the “Throughput” column indicates the maximum value that can be achieved for each platform.

Depending on the platform, throughputs range from 4.9 Mbps to 53.7 Mbps on a single processor core. From an energy perspective, the energy consumption per bit transmitted varies between 112 ηj and 702 ηj , depending on the platform. The unexpected superiority of the ARM architecture in this context can be attributed to the *vmlal* instruction, which efficiently performs fixed-point multiplication and accumulation operations, a feature that is absent in INTEL cores. In addition, $\mathcal{P}_2$ has a large L1 memory cache, which

TABLE 2. Evaluation of characteristics of the NN_1 and NN_2 FPGA-based implementations for the selected quantization format with different parallelization factors - FPGA device references are XILINX Virtex-7 (xc7vx485t-ffg1761-2) and Artix-7 (xc7a100t-csg324-1).

FPGA device	Parallelism (r, q)	LUTs	FFs	DSPs	BRAMs	Latency (cycles)	Critical path	Freq. @3000i/s.	Throughput (@fmax)	Energy/bit (@fmax)
Virtex-7	(1,1)	3341	3172	36	60	21162	3.40 ηs	215 ms	3.48 Mbps	39.1 ηj
	(8,2)	5546	8745	138	160	3658	3.34 ηs	37 ms	20.54 Mbps	2.5 ηj
Artix-7	(1,1)	3251	2984	32	62	21153	8.43 ηs	534 ms	1.40 Mbps	88.9 ηj
	(8,4)	6017	8661	113	161	3539	8.79 ηs	93 ms	8.06 Mbps	6.1 ηj

LUT: look up table, FF: flip flop, DSP: digital signal processor and BRAM: block RAM.

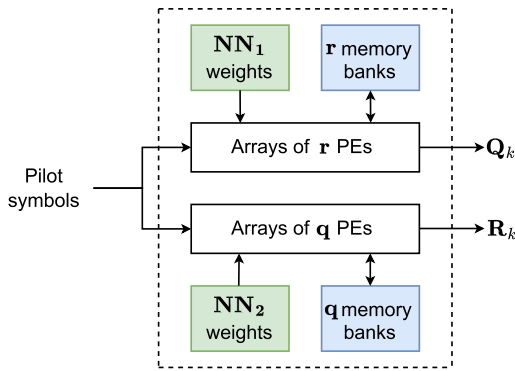


FIGURE 8. Overview of the designed hardware architecture to execute in real-time NN_1 and NN_2 on FPGA device.

offers better performance in compact neural network inference, further increasing the difference.

The proposed approach for estimating channel parameters, once implemented on these target CPUs, far exceeds the application real-time constraints as expected, even on the low-cost $\mathcal{P}_1$ target, with low-energy cost per bit.

B. HARDWARE-BASED IMPLEMENTATIONS

In highly energy-constrained systems, the use of digital architectures implemented on FPGA platforms is common practice, particularly in military applications. Numerous studies have addressed the deployment of neural networks for these hardware targets. This is largely due to the fact that the computations performed within the layers of the neural networks are highly parallelizable. Consequently, similar to the CPU experiments, we selected two platforms respectively composed of Xilinx Artix-7 and Virtex-7 FPGAs to represent the low-end and mid-range systems.

To prototype the architectures at the RTL level depicted in Figure 8 and evaluate their performance on the FPGA, we leveraged the XILINX Vitis HLS tool [54]. We developed behavioral models in C which manipulate fixed-point data using the *ap_int* type. These models are enhanced with synthesis directives to assist in the generation of architectures and thus obtain different trade-offs between complexity and throughput. The results obtained from these experiments are summarized in Table 2. Only the two best trade-off

architectures are reported herein. It should be noted that, because the computational complexity of the NN_1 and (NN_2) networks is different, the number of PEs allocated, denoted by (r, q), is not symmetric.

All the implementations presented in Table 2, meet the throughput requirements imposed by the target application. As expected, increasing the degree of parallelism led to a reduction in latency at the cost of a greater use of hardware resources. Data throughputs vary between 1.4 and 20.5 Mbps depending on the allocated PEs. Power consumption was estimated using the XPower tool. The estimated values ranged from 2 to 80 ηj when the system operated at the maximum frequency. It should be noted that, depending on the configuration, 25% to 50% of the power consumption comes from the static consumption of the FPGA part.

C. EXPERIMENTATION REVIEW

The experimental results from the industrial use case reported in this section demonstrate that the proposed NN-enhanced estimator can be implemented in real-time in constrained embedded systems. The throughput achieved with FPGA technology when low parallelization factors are used ranges from 1 to several tens of Mbps for power consumption below 100 nj per bit. On multi-core CPU targets, even though the energy consumption is significantly lower (up to 20 $\times$), the proposed approach achieves the targeted throughput even on a Cortex-A72 target, highlighting the low computational complexity of our solution and its real applicability in real systems.

VI. CONCLUSION

In this paper, a hybrid low-complexity channel estimation algorithm for slow-fading channels was proposed. This approach combines a linear Kalman filter with two neural networks to estimate the process and observation noise covariance matrices at each time step. The proposed NN-enhanced estimator achieved significant performance improvements over reference algorithms. For instance, a gain of 5 dB over LS at a low SNR and 3 dB at a high SNR, and a consistent 2 dB gain over the linear Kalman filter at both low and high SNR. To assess the low-complexity and real-time features of the proposed solution, it was implemented on both multi-core CPU and FPGA devices using common

design methodologies. Detailed fixed-point implementations reduce energy consumption and improve processing throughput without impacting channel estimation performance to reach throughput from one to tens of Mbps thereby meeting targeted application constraints.

REFERENCES

- [1] J. Proakis and M. Salehi, *Digital Communications*, 5th ed., New York, NY, USA: McGraw-Hill, 2008.
- [2] S. Qureshi, "Adaptive equalization," *Proc. IEEE*, vol. 20, no. 2, pp. 1349–1387, Jun. 1982.
- [3] S. Coleri, M. Ergen, A. Puri, and A. Bahai, "Channel estimation techniques based on pilot arrangement in OFDM systems," *IEEE Trans. Broadcast.*, vol. 48, no. 3, pp. 223–229, Sep. 2002.
- [4] M. B. Sutar and V. S. Patil, "LS and MMSE estimation with different fading channels for OFDM system," in *Proc. Int. Conf. Electron., Commun. Aerosp. Technol. (ICECA)*, vol. 1, Coimbatore, India, Apr. 2017, pp. 740–745.
- [5] M. A. Mohammadi, M. Ardabilipour, B. Moussakhani, and Z. Mobini, "Performance comparison of RLS and LMS channel estimation techniques with optimum training sequences for MIMO-OFDM systems," in *Proc. 5th IFIP Int. Conf. Wireless Opt. Commun. Netw. (WOCN)*, Surabaya, Indonesia, May 2008, pp. 1–5.
- [6] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960.
- [7] P. Banelli, R. C. Cannizzaro, and L. Rugini, "Data-aided Kalman tracking for channel estimation in Doppler-affected OFDM systems," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, Honolulu, HI, USA, Apr. 2007, pp. 133–136.
- [8] C. Ramesh and V. Vaidehi, "IMM based Kalman filter for channel estimation in UWB OFDM systems," in *Proc. Int. Conf. Signal Process., Commun. Netw.*, Chennai, India, Feb. 2007, pp. 320–325.
- [9] M. L. D. Wong and A. K. Nandi, "Automatic digital modulation recognition using artificial neural network and genetic algorithm," *Signal Process.*, vol. 84, no. 2, pp. 351–365, Feb. 2004.
- [10] I. Ortuno, M. Ortuno, and J. A. Delgado, "Error correcting neural networks for channels with Gaussian noise," in *Proc. IJCNN Int. Joint Conf. Neural Netw.*, vol. 4, Baltimore, MD, USA, Jun. 1992, pp. 295–300.
- [11] K. Hiray and K. V. Babu, "A neural network based channel estimation scheme for OFDM system," in *Proc. Int. Conf. Commun. Signal Process. (ICCCSP)*, Apr. 2016, pp. 438–441.
- [12] L. Ge et al., "Deep neural network based channel estimation for massive MIMO-OFDM systems with imperfect channel state information," *IEEE Syst. J.*, vol. 16, no. 3, pp. 4675–4685, Sep. 2022.
- [13] P. Jiang, C.-K. Wen, S. Jin, and G. Y. Li, "Dual CNN-based channel estimation for MIMO-OFDM systems," *IEEE Trans. Commun.*, vol. 69, no. 9, pp. 5859–5872, Sep. 2021.
- [14] T. Liu, M. Cui, G. Zhang, Q. Wu, X. Chu, and J. Zhang, "3D trajectory and transmit power optimization for UAV-enabled multi-link relaying systems," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 1, pp. 392–405, Mar. 2021.
- [15] W. Kim, Y. Ahn, J. Kim, and B. Shim, "Towards deep learning-aided wireless channel estimation and channel state information feedback for 6G," *J. Commun. Netw.*, vol. 25, no. 1, pp. 61–75, Feb. 2023.
- [16] E. Balevi, A. Doshi, A. Jalal, A. Dimakis, and J. G. Andrews, "High dimensional channel estimation using deep generative networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 18–30, Jan. 2021.
- [17] A. K. Gizzini, M. Chafii, A. Nimr, and G. Fettweis, "Deep learning based channel estimation schemes for IEEE 802.11p standard," *IEEE Access*, vol. 8, pp. 113751–113765, 2020.
- [18] A. Siebert, G. Ferré, B. Le Gal, and A. Fourny, "The smart Kalman filter: A deep learning-based approach for time-varying channel estimation," in *Proc. IEEE 34th Annu. Int. Symp. Pers., Indoor Mobile Radio Commun. (PIMRC)*, Toronto, ON, Canada, Sep. 2023, pp. 1–6.
- [19] K. D. Solomon Raj and I. M. Krishna, "Kalman filter based target tracking for track while scan data processing," in *Proc. 2nd Int. Conf. Electron. Commun. Syst. (ICECS)*, Coimbatore, India, Feb. 2015, pp. 878–883.
- [20] A. Almamori and S. Mohan, "Estimation of channel state information for massive MIMO based on received data using Kalman filter," in *Proc. IEEE 8th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Las Vegas, NV, USA, Jan. 2018, pp. 665–669.
- [21] Y. Liao, G. Sun, Z. Cai, X. Shen, and Z. Huang, "Nonlinear Kalman filter-based robust channel estimation for high mobility OFDM systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 11, pp. 7219–7231, Nov. 2021.
- [22] H. A. P. Blom and Y. Bar-Shalom, "The interacting multiple model algorithm for systems with Markovian switching coefficients," *IEEE Trans. Autom. Control*, vol. AC-33, no. 8, pp. 780–783, Aug. 1988.
- [23] H. Coskun, F. Achilles, R. DiPietro, N. Navab, and F. Tombari, "Long short-term memory Kalman filters: Recurrent neural estimators for pose regularization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 5525–5533.
- [24] C. Chen, C. X. Lu, B. Wang, N. Trigoni, and A. Markham, "DynaNet: Neural Kalman dynamical model for motion estimation and prediction," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 12, pp. 5479–5491, Dec. 2021.
- [25] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. G. van Sloun, and Y. C. Eldar, "KalmanNet: Neural network aided Kalman filtering for partially known dynamics," *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.
- [26] M. A. K. Raiaan et al., "A review on large language models: Architectures, applications, taxonomies, open issues and challenges," *IEEE Access*, vol. 12, pp. 26839–26874, 2024.
- [27] A. Younesi, M. Ansari, M. Fazli, A. Ejlali, M. Shafique, and J. Henkel, "A comprehensive survey of convolutions in deep learning: Applications, challenges, and future trends," *IEEE Access*, vol. 12, pp. 41180–41218, 2024.
- [28] Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A survey of convolutional neural networks: Analysis, applications, and prospects," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 12, pp. 6999–7019, Dec. 2022.
- [29] J. Niu, W. Zhang, C. J. Xue, and N. Guan, "RTIL: Real-time inference of large language models on memory-constrained GPU devices," in *Proc. IEEE 30th Int. Conf. Embedded Real-Time Comput. Syst. Appl. (RTCSA)*, Aug. 2024, pp. 21–30.
- [30] B. Del Monte and R. Prodan, "A scalable GPU-enabled framework for training deep neural networks," in *Proc. 2nd Int. Conf. Green High Perform. Comput. (ICGHPC)*, Nagercoil, India, Feb. 2016, pp. 1–8.
- [31] T. Garbay et al., "CNN inference costs estimation on microcontrollers: The EST primitive-based model," in *Proc. 28th IEEE Int. Conf. Electron., Circuits, Syst. (ICECS)*, Dubai, United Arab Emirates, Nov. 2021, pp. 1–5.
- [32] M. Sailesh, K. Selvakumar, and N. Prasanth, "A novel framework for deployment of CNN models using post-training quantization on microcontroller," *Microprocessors Microsystems*, vol. 94, Oct. 2022, Art. no. 104634.
- [33] H. Hussain, P. S. Tamizharasan, and P. K. Yadav, "LCRM: Layer-wise complexity reduction method for CNN model optimization on end devices," *IEEE Access*, vol. 11, pp. 66838–66857, 2023.
- [34] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," 2016, *arXiv:1602.07360*.
- [35] S. Ünalacak and J. J. Singaravadevelu, "Energy efficiency of convolutional neural network inference on FPGAs and accelerated GPUs," Master's thesis, Dept. Comput. Sci. Eng., University Of Gothenburg, Gothenburg, Sweden, 2021.
- [36] Y. Hu, Y. Liu, and Z. Liu, "A survey on convolutional neural network accelerators: GPU, FPGA and ASIC," in *Proc. 14th Int. Conf. Comput. Res. Develop. (ICCRD)*, Shenzhen, China: IEEE, Jan. 2022, pp. 100–107.
- [37] J. Li et al., "Large language model inference acceleration: A comprehensive hardware perspective," 2025, *arXiv:2410.04466*.
- [38] X. Chang, H. Pan, D. Zhang, Q. Sun, and W. Lin, "A memory-optimized and energy-efficient CNN acceleration architecture based on FPGA," in *Proc. IEEE 28th Int. Symp. Ind. Electron. (ISIE)*, Vancouver, BC, Canada: IEEE, Jun. 2019, pp. 2137–2141.
- [39] M. Haileselassie, S. R. Hasan, F. Khalid, F. A. Wad, and M. Shafique, "FPGA-based convolutional neural network architecture with reduced parameter requirements," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Florence, Italy, May 2018, pp. 1–5.
- [40] S. Basalama, A. Sohrabizadeh, J. Wang, L. Guo, and J. Cong, "FlexCNN: An end-to-end framework for composing CNN accelerators on FPGA," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 2, pp. 1–32, Mar. 2023.
- [41] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches," 2014, *arXiv:1409.1259*.

- [42] T. Rappaport, *Wireless Communications: Principles and Practice*. Upper Saddle River, NJ, USA: Prentice-Hall, 1996.
- [43] C. Hide, T. Moore, and M. Smith, "Adaptive Kalman filtering for low-cost INS/GPS," in *Proc. Int. Tech. Meeting Satell. Division Inst. Navigat. (ION GPS)*, vol. 56, Sep. 2003, pp. 143–152.
- [44] H. M. D. Kabir, A. Khosravi, M. A. Hosen, and S. Nahavandi, "Neural network-based uncertainty quantification: A survey of methodologies and applications," *IEEE Access*, vol. 6, pp. 36218–36234, 2018.
- [45] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 2704–2713.
- [46] M. Li et al., "Contemporary advances in neural network quantization: A survey," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Yokohama, Japan, Jun. 2024, pp. 1–10.
- [47] A. N. Mazumder et al., "A survey on the optimization of neural network accelerators for micro-AI on-device inference," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 4, pp. 532–547, Dec. 2021.
- [48] S. Wang and Y. Kang, "Gradient distribution-aware INT8 training for neural networks," *Neurocomputing*, vol. 541, Jul. 2023, Art. no. 126269.
- [49] A. N. Kuzmin, M. V. Baalen, Y. Ren, M. Nagel, J. W. T. Peters, and T. Blankevoort, "FP8 quantization: The power of the exponent," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2022, pp. 14651–14662.
- [50] M. D. Rice, S. B. Seidman, and P. Y. Wang, "A formal model for SIMD computation," in *Proc. 2nd Symp. Frontiers Massively Parallel Comput.*, Fairfax, VA, USA, Oct. 1988, pp. 601–607.
- [51] L. Huang, Y. Lü, S. Ma, N. Xiao, and Z. Wang, "SIMD stealing: Architectural support for efficient data parallel execution on multicores," *Microprocessors Microsyst.*, vol. 65, pp. 136–147, Mar. 2019.
- [52] E. Limonova, D. Ilin, and D. Nikolaev, "Improving neural network performance on SIMD architectures," *Proc. SPIE*, vol. 9875, Dec. 2015, Art. no. 98750.
- [53] S.-J. Lee, S.-S. Park, and K.-S. Chung, "Efficient SIMD implementation for accelerating convolutional neural network," in *Proc. 4th Int. Conf. Commun. Inf. Process.*, Qingdao, China, Nov. 2018, pp. 174–179.
- [54] J. Cong et al., "FPGA HLS today: Successes, challenges, and opportunities," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 15, no. 4, pp. 1–42, Aug. 2022.



ANTOINE SIEBERT received the M.S. degree in telecommunication engineering and signal processing from the ENSEIRB-MATMECA Engineering School, Bordeaux, France, in 2021, and the Ph.D. degree through a CIFRE program in collaboration with Thales Six GTS France and the IMS Laboratory, Bordeaux, in 2025. His research interests include the integration of artificial intelligence techniques into the physical layer of wireless communication systems, with a particular emphasis on channel estimation. His work aimed to develop efficient and practically implementable AI-based solutions that enhance the channel estimation performance while maintaining a balance between accuracy and computational complexity. He has been the author of three peer-reviewed publications at international conferences.



BERTRAND LE GAL received the M.S. and Ph.D. degrees from the University of South Brittany, Lorient, France, in 2002 and 2005, respectively. In 2005, he joined the IRISA/INRIA Laboratory, University of Rennes 1, and the ENSSAT Graduate Engineering School, Lannion, France, as a Teaching Assistant. From 2006 and 2023, he was an Associate Professor with the IMS Laboratory, ENSEIRB-MATMECA Engineering School, France. In 2023, he received the accreditation to supervise doctoral research from the University of Bordeaux, France. He is the author or co-author of 34 peer-reviewed publications in international journals and more than 100 peer-reviewed articles in international conferences. Then, in September 2023, he became an Associate Professor with IRISA/INRIA Laboratory of the University of Rennes. His research focused on algorithm-architecture-matching of ECC decoders on both hardware (ASIC/FPGA) and software targets (CPU/DSP/GPU). His research interests include digital communication system implementation and RISC-V processor design under performance and security constraints.



GUILAUME FERRE received the Ph.D. degree in digital communications and signal processing from Limoges University of Technology, in 2006. From 2006 to 2008, he was a Postdoctoral Researcher with Limoges XLIM Laboratory and then at the IMS Laboratory, Bordeaux. Since 2021, he is currently a Full Professor with ENSEIRB-MATMECA, Engineering School, Bordeaux INP. After several administrative responsibilities in the Telecommunications Department, ENSEIRB-MATMECA, He is the Director of Industrial Relations with ENSEIRB-MATMECA. He conducts out his research activities within the IMS Laboratory in the "signal and image" team. These fields of research concern the circuits and systems for digital communications, including: signal processing and digital communications, digital enhancement for wideband power amplifiers and time interleaved analog to digital converters. He is the author of more than 170 papers published in international journals and conferences. He is also the author of 12 patents. He supervises six Ph.D. students with a significant part of industrial research activities. He is a member of several Technical Program Committees. He is the Principal Investigator (PI) of many national and international projects, and at the local level he is responsible for two research activities related to the IoT, including one to investigate on the smart campus.



AURÉLIEN FOURNY received the M.S. degree in electrical engineering and embedded systems from the ENSEIRB-MATMECA Engineering School, Bordeaux, France, in 2017. He is currently an FPGA Project Manager with Thales, France. His work focuses on digital communications and FPGA implementations. He contributes to the software defined radio development for highly-constrained applications. In this context, he has supervised a thesis in partnership with IMS Bordeaux, France, to evaluate the capabilities of implementing IA in low footprint hardware (SWAP).

...